



COMSATS University Islamabad, Abbottabad Campus

Department of Computer Science

Software Reverse Engineering (CSE 327)

Assignment 3 (CLO 3)

Tool 1: Selenium

Selenium is primarily used for **automated testing** of web applications, but it can also play a valuable role in **reverse engineering legacy web systems**—especially when **documentation is missing**, **source code is unavailable**, or **business logic is only accessible via the UI**. Selenium helps you automate actions (clicks, inputs) in the browser, so you can observe how the system behaves and **extract the hidden logic** from the user interface.

Prerequisites:

Step 0: – What You Need before You Start

Here's what you need to set up:

Step 1: A computer with:

- Any OS: Windows, Mac, or Linux
- Internet access
- Admin rights (to install Python, browser drivers)

Step 2: Python Installed

Selenium works with many languages, but Python is easiest.

To install Python:

- Go to: <https://www.python.org/downloads/>
- Download and install Python 3.x
- Confirm it's installed:
Open a terminal (CMD) and type:
 - python --version

Step 3: Install Selenium Library

Install the Selenium package in Python.

pip install selenium

Step 4: Browser + WebDriver

You need:

- A browser (e.g., **Google Chrome**)
- A matching **ChromeDriver**

Download ChromeDriver:

- Go to: <https://sites.google.com/chromium.org/driver/>
- Match it with your Chrome version (check under chrome://settings/help)
- Place the chromedriver.exe in the same folder as your Python script or add it to your system PATH.

Step 5: Any Text Editor or IDE

You can write your Python scripts in:

- **VS Code**
- **PyCharm**

- **Notepad++**
- Or even Notepad (less preferred)

Full Step-by-Step Process

Step 1: Understand the Legacy System

- Open the legacy system in the browser.
- Note key actions like:
 - Login
 - Filling a form
 - Submitting data
 - Searching for records

Create a list like this:

Feature	Page URL	Action
Login	/login	Enter username/password
Add Invoice	/add-invoice	Fill form, save
Search Orders	/search-orders	Enter keywords, click search

Step 2: Open Selenium and Start Automating

Here's a basic Python script to launch the browser and open the system:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

# Start browser
driver = webdriver.Chrome() # Make sure ChromeDriver is available
driver.get("http://legacy-system-url/login")

# Let it load
time.sleep(2)
```

Step 3: Simulate User Actions (Click, Input, Submit)

For example, login to the system:

```
# Fill in the login form
driver.find_element(By.NAME, "username").send_keys("admin")
driver.find_element(By.NAME, "password").send_keys("admin123")

# Click the login button
driver.find_element(By.ID, "loginBtn").click()

time.sleep(3)
```

Step 4: Observe Changes

- Capture the HTML after actions
- Take screenshots

```
# Save HTML of the page after login
with open("after_login.html", "w", encoding='utf-8') as f:
    f.write(driver.page_source)
```

```
# Save a screenshot
driver.save_screenshot("login_screen.png")
```

Step 5: Repeat for Other Pages

Follow the same logic:

- Go to another page
- Fill form
- Click buttons
- Capture response

```
driver.get("http://legacy-system-url/add-invoice")
```

```
# Example: fill a form
driver.find_element(By.NAME, "amount").send_keys("100")
driver.find_element(By.ID, "submitBtn").click()
```

```
# Capture page again
driver.save_screenshot("invoice_created.png")
```

Step 6: Document What You Observe

You now have:

- Screenshots
- HTML of the page before and after actions
- Field names
- Button IDs
- URLs for different features

Use this info to create:

- Flowcharts
- Process diagrams
- Functional documentation

Step 7: (Optional) Use Browser DevTools

Use Chrome Developer Tools (F12) to:

- Inspect hidden fields or JavaScript
- View network requests
- Understand which APIs are being called when buttons are clicked

Example Outcome

You might discover:

- Login requires username, password, and a hidden token
- Invoice form must have customer ID, amount, and tax
- Some fields are validated using JavaScript

Then you can document:

Page	Fields	Rules
Login	username, password	Token is required (found via HTML)
Add Invoice	customerID, amount, tax	Amount > 0, tax auto-calculated

Tool 2: Docker

Docker is a **containerization tool** that helps package applications and their dependencies into isolated containers. When you're working with a **legacy system**, Docker can be used in **reverse engineering** to help **replicate, analyze, and modernize** the system—especially when the original environment is outdated or undocumented. Use Docker to **capture, reproduce, and explore** how a legacy system works—without breaking the original setup.

Docker is a **containerization tool** that helps package applications and their dependencies into isolated containers. When you're working with a **legacy system**, Docker can be used in **reverse engineering** to help **replicate, analyze, and modernize** the system—especially when the original environment is outdated or undocumented.

Prerequisites

Step 0:– What You Need

Tool	Description
Docker	Install Docker Desktop from https://www.docker.com/products/docker-desktop
Access	You need access to the legacy app's files or server (even partially: code, configs, binaries, logs)
Basic Knowledge	Terminal/Command Line, Linux basics (if possible)

Step-by-Step Process

Step 1: Understand the Legacy Environment

Before doing anything:

- Ask: Where is the app hosted (Linux, Windows, on-premise)?
- What components are involved?
 - Web server (Apache, Nginx)?
 - Database (MySQL, Oracle)?
 - Any scripts or compiled code (PHP, Java, .exe)?

Example:

Legacy system includes:

- PHP app
- Apache server
- MySQL 5.5 database
- Running on Ubuntu 14.04

Step 2: Copy the Environment

Try to get these from the legacy system:

- Application files (e.g., PHP, HTML, JS, etc.)
- Config files (httpd.conf, .env, my.cnf, etc.)
- Database dump (e.g., .sql file)

Step 3: Create a Dockerfile (to Rebuild Environment)

A Dockerfile defines the OS and environment needed.

Example Dockerfile for a PHP-Apache legacy app:

```
FROM php:5.6-apache # match legacy PHP version
```

```
COPY . /var/www/html/ # copy app files
```

```
RUN docker-php-ext-install mysqli # install MySQL driver
```

Step 4: Create a docker-compose.yml (for full stack)

If the app uses a database or multiple services, use **Docker Compose** to define everything.

Example docker-compose.yml:

```
version: '3'
```

```
services:
```

```
  web:
```

```
    build: .
```

```
    ports:
```

```
      - "8080:80"
```

```
    volumes:
```

```
      - .:/var/www/html
```

```
  db:
```

```
    image: mysql:5.5
```

```
    environment:
```

```
      MYSQL_ROOT_PASSWORD: root
```

```
      MYSQL_DATABASE: legacydb
```

```
    ports:
```

```
      - "3306:3306"
```

Step 5: Add a Database Dump (Optional)

Put your .sql file (database dump) in the db service using a Docker volume or command:

```
volumes:
```

```
  - ./init.sql:/docker-entrypoint-initdb.d/init.sql
```

Step 6: Run the Legacy System in Docker

In the terminal, go to your project folder and run:

```
docker-compose up --build
```

This will:

- Build the PHP-Apache environment
- Start MySQL
- Load your legacy app
- Expose it at: <http://localhost:8080>

Step 7: Explore the Running System

Once it's up, now you can:

Action	Use it to...
View app in browser	Test how the legacy system behaves
Modify files	Test impact of code/config changes
Log outputs	Check logs for hidden logic or errors
Run in isolation	No risk of breaking the real server

Step 8: Reverse Engineer Behavior

While inside the container, you can now:

- View installed packages (dpkg -l, php -v)
- Explore file structures
- Trace business logic (by following the code)
- Use tools like tcpdump or curl to analyze internal APIs

Step 9: Document Findings

As you explore:

- Write down environment details (OS, dependencies)
- Note any hardcoded paths or credentials
- Record app behavior based on input/output

You can later use this info to **migrate or refactor** the system.

Example Folder Structure

legacy-system/

```
├── Dockerfile
├── docker-compose.yml
├── init.sql
├── src/
│   ├── index.php
│   └── config.php
```

Tool 3: Apache Nifi

Apache **NiFi** is a powerful data flow automation tool used to collect, route, transform, and deliver data. When you're reverse engineering a **legacy system**, NiFi can help **visually trace how data flows** in and out of the system—especially when dealing with legacy databases, APIs, files, or message queues. Use Apache NiFi to reverse engineer data flow in a legacy system by **visually mapping, monitoring, and analyzing** how data moves across systems.

Prerequisites: Step 0

Item	Description
Java Installed	NiFi runs on Java (JDK 8+ recommended)
Apache NiFi	Download from: https://nifi.apache.org/download.html

Item	Description
Basic Concepts	Understand source/destination systems (e.g., DB, FTP, APIs)
Access	You need access to where the data lives: file system, DB, API, etc.

Step-by-Step process:

Step 1: Download and Start NiFi

1. Download NiFi (binary release):
<https://nifi.apache.org/download.html>
2. Unzip it and start:
3. `./bin/nifi.sh start`
4. Open in browser:
5. `http://localhost:8080/nifi`

Step 2: Understand Legacy Data Sources

Identify what the legacy system uses:

- Files (CSV, XML, JSON)
- Relational DB (MySQL, Oracle, etc.)
- REST APIs
- Message queues (Kafka, JMS)

Example:

- Legacy app stores data in `old_data.csv`
- Sends updates via HTTP to a client system
- Uses MySQL 5.5 as backend

Step 3: Use NiFi Processors to Tap Into the Data

Create a **data flow pipeline** using NiFi's drag-and-drop UI.

Data Source	NiFi Processor
Files	GetFile, ListFile
DB	ExecuteSQL, QueryDatabaseTable
HTTP API	InvokeHTTP, ListenHTTP
FTP/SFTP	GetSFTP, GetFTP

Step 4: Trace Data Movement and Structure

As you pull data from legacy systems:

- Use **LogAttribute** to print what data looks like.
- Use **PutFile** or **PutDatabaseRecord** to capture it.
- Use **RouteOnAttribute** or **EvaluateJsonPath** to break down fields.

Example Flow:

GetFile (CSV) → SplitText → ConvertRecord → LogAttribute → PutDatabaseRecord

This lets you:

- Read the file
- Break it into rows
- Convert it to structured form
- Log and observe the structure
- Push it into a modern database (optional)

Step 5: Analyze and Document the Logic

NiFi allows you to **inspect**:

- File structure
- Database schema
- API behavior (input/output)
- Triggers or schedules

Document:

- What kind of data is being processed?
- Where is it coming from?
- How is it transformed or filtered?
- Where is it going?

This becomes your **functional and data flow documentation** of the legacy system.

Step 6: Add Debugging or Testing Flows

NiFi has useful tools for debugging:

- Use **LogAttribute** to see flowfile content
- Use **ReplaceText** or **UpdateAttribute** to simulate transformations
- Use **PutFile** to export all intermediate outputs

This lets you test:

- How data is structured
- Whether flows break under malformed input
- Hidden behaviors in old systems (like triggers)

Step 7: Export and Share Your Flow

You can export your NiFi flow as a template:

- Right-click the canvas → **Create Template**
- Save the .xml file
- Share with others to replicate the reverse-engineered setup

Bonus: NiFi Registry for Versioning

Install [Apache NiFi Registry](#) to:

- Version control your flows
- Compare changes
- Document history of discovery

4. Design Pattern Implementation:

Smart University Learning Management System (LMS)

A university is developing a **Learning Management System (LMS)** for **Students, Teachers, Administrators, and Parents**. Since each user has different responsibilities, the system should provide a customized dashboard and features based on the user's role.

Teachers can upload lecture materials, create quizzes, grade assignments, record attendance, and post announcements. Students can access course materials, submit assignments, attempt quizzes, view grades, and attend online classes. Administrators manage users and courses, while parents can monitor their child's academic progress and attendance.

The university also plans to integrate new services such as an **AI Tutor, Video Conferencing, Digital Library, and Online Examination System** in the future. These features should be added without requiring major changes to the existing system.

Whenever a teacher posts an announcement, uploads grades, or updates assignment results, all relevant users (students, parents, and the course coordinator) should automatically receive notifications.

Tasks:

- Analyze the scenario and identify the key software design challenges.
- Select **two appropriate design patterns** for the system.
- Justify why the selected design patterns are appropriate.
- Draw the **UML Class Diagram** for your proposed solution.
- Implement the selected design patterns using **Java, C#, C++, or Python**.

Note:

- **Select any project relevant to these tools.**
- **Install these projects completely in your laptop.**
- **Then implement these tools and add the results in the form of screenshots in your assignment.**
- **It is a group assignment. But evaluation will be on the basis of presenting yours assignment in the class.**

End
